



```
In [1]: import pandas as pd
import re
import nltk
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_
nltk.download('stopwords')
```

```
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Unzipping corpora/stopwords.zip.
```

Out[1]: True

```
In [4]: df = pd.read_csv('/content/news.csv', index_col=0)
print("First 5 rows of the dataset:")
print(df.head())
X = df['text']
y = df['label']
print(f"\nTotal dataset size: {len(df)} samples")
print("\nClass distribution:")
print(df['label'].value_counts())
```

First 5 rows of the dataset:

	title \		text	label
8476	You Can Smell Hillary's Fear		Daniel Greenfield, a Shillman Journalism Fello...	FAKE
10294	Watch The Exact Moment Paul Ryan Committed Pol...		Google Pinterest Digg Linkedin Reddit Stumbleu...	FAKE
3608	Kerry to go to Paris in gesture of sympathy		U.S. Secretary of State John F. Kerry said Mon...	REAL
10142	Bernie supporters on Twitter erupt in anger ag...		– Kaydee King (@KaydeeKing) November 9, 2016 T...	FAKE
875	The Battle of New York: Why This Primary Matters		It's primary day in New York and front-runners...	REAL

Total dataset size: 6335 samples

Class distribution:

label

REAL 3171

FAKE 3164

Name: count, dtype: int64

```
In [5]: stemmer = PorterStemmer()
stop_words = set(stopwords.words('english'))

def preprocess_text(text):
    """
    This function performs several preprocessing steps on the input text.
```

```

"""
text = text.lower()
text = re.sub(r'^a-zA-Z\s', '', text)
text = ' '.join([stemmer.stem(word) for word in text.split() if word not in stopwords])
return text

print("Applying text preprocessing... This may take a moment.")
X_processed = X.apply(preprocess_text)
print("Preprocessing complete.")
print("\nOriginal text (first 500 chars):")
print(X.iloc[0][:500])
print("\nProcessed text (first 500 chars):")
print(X_processed.iloc[0][:500])

```

Applying text preprocessing... This may take a moment.
Preprocessing complete.

Original text (first 500 chars):

Daniel Greenfield, a Shillman Journalism Fellow at the Freedom Center, is a New York writer focusing on radical Islam. In the final stretch of the election, Hillary Rodham Clinton has gone to war with the FBI. The word “unprecedented” has been thrown around so often this election that it ought to be retired. But it’s still unprecedented for the nominee of a major political party to go war with the FBI. But that’s exactly what Hillary and her people have done. Coma patients just waking up now

Processed text (first 500 chars):

daniel greenfield shillman journal fellow freedom center new york writer focus radical islam final stretch elect hillari rodham clinton gone war fbi word unprecedented thrown around often elect ought retir still unprec nomine major polit parti go war fbi that exactli hillari peopl done coma patient wake watch hour cnn hospital bed would assum fbi director jame comey hillari oppon elect fbi attack everyone obama cnn hillari peopl circul letter attack comey current media hit piecelambast target trump woul

```

In [6]: vectorizer = CountVectorizer()
X_features = vectorizer.fit_transform(X_processed)
print(f"Feature matrix shape: {X_features.shape}")
print(f"Number of unique words (features) in the vocabulary: {X_features.get_feature_names_out().shape}")
print(f"20 sample feature names (words):")
print(vectorizer.get_feature_names_out()[1000:1020])

```

Feature matrix shape: (6335, 58796)
Number of unique words (features) in the vocabulary: 58796

20 sample feature names (words):

['ajk' 'ajnyuedu' 'ak' 'aka' 'akai' 'akajsaid' 'akam' 'akarlsmith' 'akash' 'akasha' 'akath' 'akbar' 'akber' 'akcakal' 'aker' 'akerlof' 'akhbar' 'akhikesh' 'akhilesh' 'akhmetov']

```

In [7]: X_train, X_test, y_train, y_test = train_test_split(X_features, y, test_size=0.2, random_state=42)

```

```
# Print the sizes of the resulting sets.
print(f"Training set size: {X_train.shape[0]} samples")
print(f"Testing set size: {X_test.shape[0]} samples")
```

Training set size: 5068 samples
Testing set size: 1267 samples

```
In [9]: model = MultinomialNB()
model.fit(X_train, y_train)
print("Model training complete.")
print(f"Model parameters: {model.get_params()}")
```

Model training complete.
Model parameters: {'alpha': 1.0, 'class_prior': None, 'fit_prior': True, 'force_alpha': True}

```
In [10]: y_pred = model.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred, pos_label='FAKE')
recall = recall_score(y_test, y_pred, pos_label='FAKE')
f1 = f1_score(y_test, y_pred, pos_label='FAKE')
conf_matrix = confusion_matrix(y_test, y_pred)

print("Model Evaluation Results:")
print(f"Accuracy: {accuracy:.4f}")
print(f"Precision: {precision:.4f}")
print(f"Recall: {recall:.4f}")
print(f"F1-Score: {f1:.4f}")
print("\nConfusion Matrix:")
print(conf_matrix)
```

Model Evaluation Results:

Accuracy: 0.8856
Precision: 0.9045
Recall: 0.8599
F1-Score: 0.8816

Confusion Matrix:

```
[[540  88]
 [ 57 582]]
```